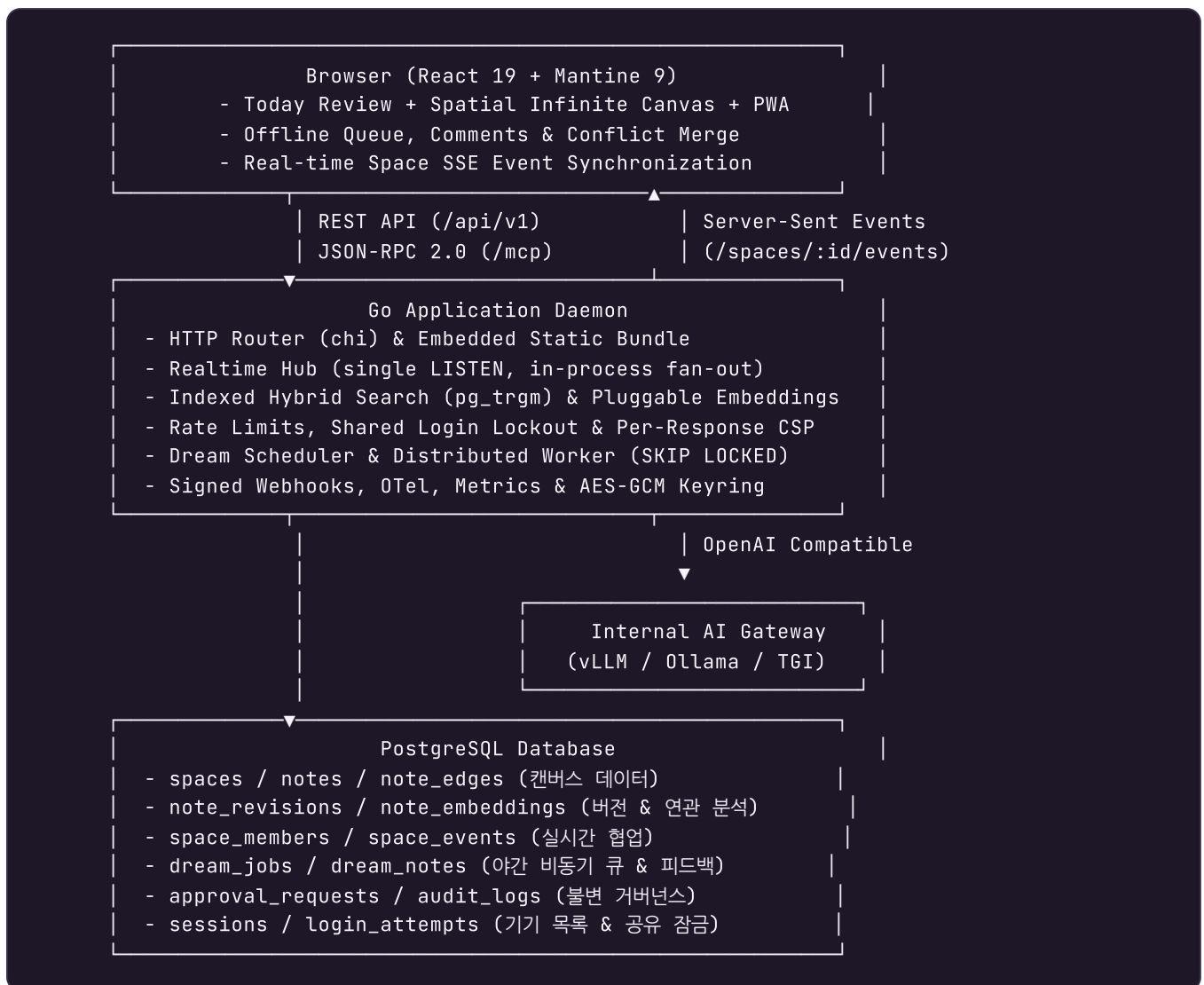


umm 실행 아키텍처 및 불변 조건 (Architecture)

umm은 단일 Go 바이너리와 단일 PostgreSQL 인스턴스만으로 오프라인 폐쇄망에서 무중단 운영이 가능하도록 설계된 **Spatial Thought Memory** 엔진입니다.

1. 시스템 아키텍처 다이어그램



2. 데이터 경계 및 도메인 분리

- **Canvas Domain:** spaces, notes, note_edges
- **Intelligence Domain:** note_revisions, note_embeddings

- **Collaboration Domain:** `space_members`, `space_events`, `notifications`, `note_comments`, `comment_mentions`
- **Identity & Access:** `users`, `sessions`, `oauth_states`, `teams`, `login_attempts`, `ai_quota_reservations`
- **Personalization & Review:** `user_preferences`, `note_reviews`, `api_keys`
- **Governance & Security:** `approval_requests`, `audit_logs`, `app_settings`
- **Dream & LLM Domain:** `dream_jobs`, `dream_notes`, `dream_sources`, `dream_feedback`, `ai_calls`, `ai_eval_cases`, `ai_eval_runs`
- **Automation & Reliability:** `webhook_subscriptions`, `webhook_deliveries`, `idempotency_records`, `product_events`

3. 핵심 아키텍처 원칙

1. 외부 의존성 없는 의미 연관성 분석 (Offline Semantic Engine)

- 연관 생각(Related Thoughts)과 클러스터링은 기본적으로 **192차원 문자 n-gram feature hashing**과 **cosine similarity**로 계산됩니다. 외부 다운로드나 API 호출이 없어 폐쇄망에서 100% 동작합니다.
- v0.8.0부터 관리자가 AI Gateway에 임베딩 모델을 지정하면 OpenAI 호환 `/v1/embeddings`를 대신 사용합니다. 모든 벡터에는 모델명과 정규화된 Gateway endpoint의 SHA-256 fingerprint로 만든 알고리즘 식별자가 함께 저장되고 **같은 알고리즘끼리만 비교**합니다. 모델 또는 Gateway를 바꾸면 같은 모델명을 유지해도 서로 다른 벡터 공간이 뒤섞이지 않고 점진적으로 다시 임베딩됩니다. API key와 timeout은 벡터 공간을 바꾸지 않으므로 fingerprint 대상에서 제외합니다. 원격 응답과 장애 뒤 비교 집합을 로컬로 통일하는 후속 pass를 저장하는 transaction은 모두 작업 시작 시점의 `ai_gateway` 설정 세대로 현재 설정 행을 shared lock 확인하므로, 두 pass 사이 또는 호출 중 설정이 바뀌어도 이전 결과가 새 Gateway의 벡터를 덮어쓸 수 없습니다. 외부 임베딩 직전에는 같은 설정 행과 대상 note·space의 `ai_excluded` 행을 다시 잠그고 응답 저장이 끝날 때까지 유지합니다. 제외 설정이 먼저 확정되면 로컬 vector로 전환하고, lease가 먼저 시작되면 정책 변경이 외부 호출 종료까지 기다리므로 point-in-time 확인 뒤 캡처한 본문이 새 정책을 넘어 전송되지 않습니다. 콘텐츠 version이 낮은 벡터도 알고리즘이 다르다는 이유만으로 최신 행을 교체하지 못합니다.
- 게이트웨이 호출이 실패하면 로컬 벡터로 내려가되 저장되는 알고리즘 이름은 로컬로 기록됩니다. 실패한 호출이 성공한 것처럼 남는 경로는 없습니다. 공간으로 범위를 제한한 검색은 접근 권한, 공간 제외, 활성 메모의 개별 `ai_excluded`를 먼저 함께 확인합니다. 하나라도 제외되면 Canvas가 정규화한 것과 같은 로컬 비교 공간을 선택하고 저장된 원격 vector가 없는 후보도 응답 본문에서 로컬 임베딩해 의미 검색을 유지합니다. 원격 검색을 선택한 경우에는 space·membership·활성 note 행을 query embedding과 검색 row 소비가 끝날 때까지 잠급니다. 제외 또는 접근 회수가 먼저 확정되면 검색어도 원격 Gateway로 보내지 않습니다. 이 임베딩 정책 lease는 Dream·AI Assist와 같은 인스턴스당 2-slot AI 전용 연결을 사용해 느린 Gateway가 request pool을 점유하지 않습니다. `enc:` Gateway API key를 복호화할 수 없으면 ciphertext를 credential로 사용하지 않고 provider 전체를 로컬로 닫습니다.

1-1. 푸시 기반 실시간 협업 (Realtime Hub)

- `space_events`의 AFTER INSERT 트리거가 `pg_notify`를 호출합니다. PostgreSQL은 알림을 커밋 시점에 전달하므로, 롤백된 변경이 다른 사람에게 보이는 상태는 만들어질 수 없습니다.

- 인스턴스마다 전용 연결 하나가 `LISTEN umm_space_events` 를 유지하고, 도착한 알림을 프로세스 안의 구독자에게 팬아웃합니다. 구독자 수와 무관하게 데이터베이스 부하는 일정합니다. 단 `pool_max_conns` 가 1 또는 2이면 전용 리스너를 시작하지 않고 기존 1초 안전 폴링을 사용해 request pool의 모든 연결을 readiness·인증·transaction 요청에 남깁니다. 상한 3부터 listener를 시작하면서 request용 두 연결을 보존합니다. 목록과 unread count를 함께 반환하는 알림 endpoint는 count와 rows를 순차 실행해 request 연결 하나만 사용하고, 짧은 Dream 채택 transaction도 같은 연결에서 권한을 확인합니다. 외부 호출 동안 유지하는 access lease는 request pool과 분리해 AI 최대 2개, 웹훅 최대 3개로 각각 제한하므로 인스턴스의 최악 연결 상한은 `pool_max_conns + 5` 입니다.
- 알림은 페이로드를 신뢰하지 않습니다. 깨어난 리더는 자신이 마지막으로 보낸 sequence 이후를 다시 조회하므로, 알림이 합쳐지거나 유실되어도 이벤트를 건너뛰지 않습니다.
- 리스너가 끊기면 지수 백오프로 재연결합니다. 연결 상태 전환 자체가 모든 SSE 구독자를 coalesced 신호로 즉시 깨우므로, 리더는 기존 30초 safety-net deadline을 기다리지 않고 cursor를 따라잡은 뒤 타이머를 1초 폴링으로 재설정합니다. 재연결 신호에서는 다시 30초 안전망으로 돌아가며 협업이 멈추는 구간이 없습니다.

1-1-1. 백엔드에 독립적인 유사도 판정

- 연관 생각·군집·검색 라벨·Dream 페어 선택은 더 이상 고정 cosine 컷오프를 쓰지 않습니다. 각 후보 집합의 관측 분포에서 **평균 위 표준편차 단위로** 기준을 잡습니다(`intelligence.SimilarityScale`).
- 이유는 측정으로 확인됐습니다. 로컬 문자 n-gram은 무관한 쌍을 0.18 부근에, 강한 일치를 0.34 부근에 놓지만, 문장 임베딩 모델은 무관한 쌍조차 0.36 위에 놓습니다. 같은 상수가 한쪽에서는 합리적이고 다른 쪽에서는 워크스페이스 전체를 통과시킵니다.
- Dream의 bridge 향도 같은 이유로 상대화했습니다. 고정 정점 0.35는 유사도 0.70 이상에서 정확히 0을 반환하므로, 임베딩 모델을 붙이면 가중치 55%의 주 신호가 통째로 사라졌습니다.
- 표본이 4개 미만이거나 분포에 퍼짐이 없으면 예전 상수로 물러섭니다. 메모가 두세 개뿐인 공간의 동작은 바뀌지 않습니다.
- 데이터셋과 채점은 `internal/intelligence/quality.go` 에 있고, CI 리포트와 관리자 화면이 **같은 코드**를 씁니다. `quality_test.go` 는 그 값을 하한으로 고정해 나빠지면 실패시킵니다. 후보 모델은 `UMM_EMBEDDING_TEST_URL / UMM_EMBEDDING_TEST_MODEL` 로 같은 기준에서 비교합니다.

0-8a. 기억에 묻기: 근거 검색과 답변

- 답변은 두 단계이고 **검색이 먼저 분리되어 있습니다**. 무엇을 읽는지가 답을 근거 있게 만들 수 있는지를 결정하고, LLM 없이 검증 가능한 유일한 절반이기 때문입니다.
- 검색만 하지 않고 **그래프를 한 걸음 따라갑니다**. 질문의 답은 질문을 던진 생각 안이 아니라 옆에 적혀 있는 경우가 많습니다(실측: 답 노트가 질문과 단어를 하나도 공유하지 않아 연결로만 도달).
- `ai_excluded` 는 세 경로 모두에서 막습니다 — 노트 단위, 공간 단위, 그리고 이웃 탐색. 검색에서 막고 이웃에서 끌어오면 더 긴 경로로 같은 유출입니다. umm은 이 결함을 v0.8.0에서 실제로 냈고, 검색 계층이 그게 잊히기 가장 쉬운 자리입니다.
- 제외된 것은 **개수를 보고합니다**. 기대보다 적은 자료로 만든 답이 완전해 보이면 안 되고, 내용을 드러내지 않고 알릴 방법은 숫자뿐입니다.
- 근거가 없으면 **모델을 호출하지 않습니다**. 근거 없는 답이 가장 그럴듯하고 가장 틀리는 경우가 그때입니다.
- **모델 미설정(412)과 게이트웨이 실패(502)를 구분합니다**. 앞의 것은 관리자가 1분이면 고치고 뒤의 것은 아닙니다.

- 답변 로직이 `internal/dream` 에 있는 이유는 게이트웨이 클라이언트·할당량·호출 로그가 거기 있기 때문입니다. 패키지 이름은 역사적인 것이고 실제로는 AI 계층입니다.

0-8b. 조력자: 살펴본 뒤 답하기

- 한 번의 검색으로 답하는 것과 **찾아보고 읽고 다시 찾아보는 것**을 나눴습니다. 뒤의 것은 다음에 무엇을 볼지 모델이 정하므로 결과가 좋아질 수도, 엉뚱해질 수도 있습니다. 그래서 두 모드가 나란히 있고 사람이 고릅니다.
- 읽기 전용을 플러그가 아니라 **도구 목록으로 구현했습니다**. 쓰기 도구를 붙여 두고 실행 직전에 권한을 확인하는 대신, 애초에 바인딩하지 않습니다. 모델이 부르고 싶어도 부를 것이 없고, 틀리려면 누군가 의도적으로 도구를 추가해야 합니다. 통합 테스트는 Go의 도구 목록이 아니라 **게이트웨이로 실제 전송된 JSON**을 검사합니다.
- 조회는 전부 `/ai/ask` 와 같은 **store** 경로를 씁니다. 접근 규칙과 AI 제의를 다시 서술하지 않아야 어긋나지 않습니다.
- 마지막 턴은 **도구 없이 보냅니다**. 도구를 쥐여 준 채 "그만 찾고 답하라"고 하면 모델은 도구를 한 번 더 부르고 사람은 빈 답을 받습니다. 만들면서 실제로 그랬습니다.
- 한 턴은 요청한 사람의 **할당량에서 하나 차감합니다**. 이 자리도 처음엔 틀렸습니다 — 빈 사용자에게 차감했고, 빈 사용자는 할당량 검사를 건너뛰므로 한 요청에 공짜 호출 6번이었습니다. 회귀 테스트가 실제 차감 건수를 셉니다.
- 조회 과정을 그대로 돌려줍니다. 살펴본 끝에 나온 답이 추측보다 나은 근거는 **어디를 봤는지 볼 수 있다는 것**뿐입니다.

0-8c. 갈래: 접어 둔 생각이 현재처럼 읽히지 않게

- 문제는 어수선했음이 아니라 이미 버린 선택지를 다시 집어드는 것입니다. 검색 결과에서 접어 둔 생각과 현재 생각은 생김새가 같습니다.
- 표시는 사람이 합니다. 한 달 동안 아무것도 안 적힌 갈래를 umm이 알아서 접지 않습니다. 침묵은 결정이 아니고, 그렇게 처리하면 잠시 멈춰 둔 일이 조용히 묻힙니다. `상충` · `질문` 과 같은 규칙입니다.
- 정리에는 이유가 필수입니다. 스키마의 CHECK 제약과 API 양쪽에서 막습니다. 이유 없는 결정 기록은 이 기능이 막으려는 망각이 한 단계 뒤로 미뤄진 것입니다.
- 표시는 두 **조회 경로 모두**에 붙습니다. 직접 일치한 생각과 연결을 따라 도달한 생각 — 뒤쪽이 더 중요합니다. 아무도 그걸 보겠다고 고르지 않았기 때문입니다.
- 표시 문구가 뜻을 결정합니다. 처음 쓴 `[보류된 갈래]` 를 실제 모델은 "아직 결정되지 않음"으로 읽어, 사용자가 버리기로 결정한 것을 미결로 보고했습니다. 지금은 채택하지 않기로 했다는 사실과 **그 이유**를 함께 씁니다. 테스트는 문구가 아니라 "보류"라는 단어 자체를 금지합니다.
- `Note` 에 컬럼을 붙이지 않았습니다. 노트를 읽는 경로가 17군데이고, 한 군데를 빠뜨리면 v0.11.0에서 실제로 낸 결함(`Backlinks` 가 `origin` · `confidence` 없이 엣지를 반환)이 그대로 재현됩니다. 갈래 정보는 필요한 곳에 따로 붙입니다.

0-8d. 렌즈: 한 갈래만 보기

- 흐리게 두되 숨기지 않습니다. 사라진 생각은 지워진 생각으로 읽히고, 캔버스가 실제보다 단출해 보입니다. 흐리게 둔 개수를 항상 함께 말합니다.
- 의미가 아니라 구조로 좁힙니다. 유사도 기반 렌즈는 뒤에 있는 임베딩만큼만 좋고, 기본 백엔드는 글자 n-gram이라 단어가 겹치는 것을 주제라고 부릅니다. 갈래와 연결은 사람이 그은 것이라 렌즈는 그 사람이 만든 구조만큼 정확하고, 백엔드가 약해도 조용히 나빠지지 않습니다.

- 흐름은 카드가 아니라 노드 바깥 요소에 겁니다. `.postit` 은 `animation: note-land ... both` 를 갖고 있고, 애니메이션의 마지막 키프레임은 캐스케이드에서 인라인 스타일을 이깁니다. 카드에 `opacity` 를 걸면 DOM에는 값이 보이지만 화면에는 한 번도 그려지지 않습니다. 실제로 그렇게 만들었고, 인라인 속성을 읽는 검증이 그걸 통과시켰습니다.
- 그래서 e2e는 `toHaveCSS` 로 계산된 스타일을 봅니다. 속성을 읽는 검사는 그리지도 않는 효과를 통과시킵니다.
- 그래프 탐색은 `web/src/lens.ts` 로 분리했습니다 — 렌즈에서 틀릴 수 있는 유일한 부분입니다. 연결은 양방향으로 따라갑니다. 화살표만 따라가면 캔버스가 바로 옆에 보여 주는 백링크가 빠집니다.

0-8e. 결정 기록

- 여섯 달 뒤의 질문은 "무엇을 적었나"가 아니라 ***"어떻게 하기로 했고 왜인가"***입니다. 왜가 가장 먼저 사라지므로 이유는 결정과 같은 화면에 둡니다.
- 표시한 것만 들어갑니다. 활동이 몰린 주를 전환점이라 부르지 않고, 조용한 갈래를 접힌 것으로 처리하지 않습니다. 결정과 추측이 섞인 기록은 기록이 없는 것보다 나쁩니다 — 필요할 때 어느 쪽인지 구분할 수 없습니다.
- 진행 중인 갈래는 제외합니다. 정해진 것이 없고, 넣으면 결정 기록이 할 일 목록이 됩니다.
- 정렬은 SQL에서 합니다. 종류별로 최근 N개를 가져와 Go에서 합치면 흔한 종류가 드문 종류의 오래된 항목을 밀어내고, 그 해에는 아무것도 결정하지 않은 것처럼 읽힙니다.
- 비어 있을 때 "표시된 것이 없다는 뜻이지 아무 일도 없었다는 뜻은 아닙니다"라고 말합니다 — `find_open_questions` 와 같은 규칙입니다.

0-8f. 접어 둔 결정이 되돌아오는 자리

- 결정 기록은 찾아봐야 보입니다. 정작 필요한 순간은 다시 쓰고 있을 때이므로, 근접 중복 한쪽이 접어 둔 갈래에 있으면 오늘의 리뷰가 결정과 이유를 함께 내놓습니다. "그거 안 하기로 했잖아"만 남으면 왜인지 모른 채 다른 이유로 다시 거절하거나 아예 거절하지 않게 됩니다.
- 여기서만 "하나로 합치기"를 뺍니다. 합치면 먼저 적은 쪽이 남고 그쪽이 접어 둔 생각입니다. 한 번 누르면 지금 쓰는 생각이 이미 거절한 갈래로 조용히 접혀 들어가고, 거절된 쪽이 남습니다. 버튼 하나로 할 일이 아닙니다.
- 정확히 한쪽만 접어 둔 갈래일 때만 표시합니다. 양쪽 다면 그냥 중복 두 개이고, 결정이 둘 다를 덮고 있습니다.
- 이 기능은 umm에서 유일한 절대 임계값(근접 중복 0.92)에 기대므로, 의미를 재는 백엔드에서만 동작하고 기본 임베딩에서는 오늘의 리뷰가 이미 그렇듯 "재지 않았음"으로 건너뛴니다.

0-9c. 질문과 답

- `notes.kind` 는 요청 본문에서 검증 없이 들어오고 있었습니다 — v0.11.0 이전의 `relation` 과 같은 결함이고, 5000자 문자열도 저장했습니다. 아무것도 읽어 오지 않아 아무도 몰랐습니다. 이제 `thought·question·idea` 어휘이고, 어휘 밖은 조용히 바꾸지 않고 거부합니다.
- 만들기와 수정 두 경로 모두 검증합니다. 한쪽만 막으면 다른 쪽이 우회로로 남고, 그게 검증되지 않은 필드가 고쳐진 뒤에도 살아남는 방식입니다.
- `answers` 가 질문을 닫습니다. `supports` · `refines` 는 가깝지만 닫지 않습니다 — 그중 하나를 답으로 치면 논쟁만 한 노트가 질문을 해결한 것처럼 보입니다.
- 질문도 답도 표시하는 것이지 추론하는 것이 아닙니다. 물음표로 끝나는 문장이 질문이 아닌 경우도, 물음표 없는 질문도 많습니다. 빈 결과는 "다 답했다"가 아니라 "표시된 게 없다"이고, 0일 때 화면에 숫자를 띄우지 않습니다.

- `attempts` 는 질문을 가리키지만 답은 아닌 연결 수입니다. 아무도 건드리지 않은 질문과 한참 논쟁하고도 결론이 안 난 질문은 다른 상황입니다.

0-9b. 기록된 상충

- `contradicts` 연결은 v0.11.0부터 만들 수 있었지만 아무것도 읽어 오지 않았습니다. `Contradictions` 가 그것을 돌려주고, 브리핑·API·MCP `find_contradictions` 가 소비합니다.
- 탐지가 아니라 기록입니다. umm은 두 노트를 읽고 모순이라고 판단하지 않습니다. 그래서 빈 결과는 "없음"이 아니라 "아무도 표시하지 않음"이고, **0일 때 화면에 숫자를 띄우지 않습니다** — "상충 0"은 워크스페이스에 모순이 없다는 뜻으로 읽힙니다.
- 상충에는 **방향**이 있습니다(source가 target을 반박). 나오는 길에 뒤바뀌면 누가 무엇을 반박하는지가 뒤집히므로 테스트로 고정합니다.
- 삭제된 생각과 접근할 수 없는 공간은 제외합니다.

0-9a. 생각 합치기

- 노트를 참조하는 테이블이 **여덟 개**라, 합치기는 캐스케이드에 맡기면 조용히 데이터를 잃습니다. 각각을 명시적으로 처리합니다: 연결·덧글·Dream 인용은 따라가고, 거절 기록은 짝이 바뀌었으므로 삭제하고, **복습 일정과 리비전은 남습니다** — 일정은 그 사람이 본 카드의 회상 타이밍이고 리비전은 사라질 행의 역사이므로, 옮기면 살아남은 생각에게 일어나지 않은 일을 기록하게 됩니다.
- 남는 쪽의 **임베딩도 지웁니다**. 내용이 바뀌었으므로 저장된 벡터는 더 이상 없는 문장을 설명합니다.
- **합쳐진 문구는 호출자가 넘깁니다**. umm은 두 노트가 거의 같다는 것은 알아도 어느 표현을 남길지는 모르고, 이어 붙이면 같은 말을 두 번 하는 노트가 됩니다.
- 자기 자신·없는 노트·빈 내용·쓰기 권한 없는 공간·**다른 공간**을 거부합니다. 마지막은 연결이 양 끝을 같은 공간에 요구하기 때문입니다.
- 같은 쌍을 반대 방향에서 합치는 두 요청이 교착하지 않도록 두 행을 **고정된 순서**로 잠급니다.

0-9. 아침 브리핑과 중복 탐지

- 브리핑은 **umm이 실제로 만들어 낸 것만** 셉니다. 모순이나 "중요도 상승" 항목이 없는 이유는 탐지하지 않기 때문입니다 — 세지 않은 항목 옆의 `0` 은 "없다"로 읽히지 "아무도 안 봤다"로 읽히지 않습니다.
- 그래서 응답에 `skipped` 가 있습니다. 빈 `duplicates` 는 백엔드에 따라 뜻이 다르고, `quiet` 는 **보고할 것도 건너뛴 것도 없을 때만** 참입니다.
- 중복 판정은 umm에서 **유일한 절대 코사인 임계값**입니다. 다른 모든 기준이 상대인 이유는 백엔드마다 "가깝다"가 다르기 때문인데, 거의 같은 글은 어떤 임베딩 공간에서도 맨 위에 오고 두 모델이 같은 지점에 둡니다(측정: bge-m3 0.943+, paraphrase-multilingual 0.954+, 그다음 등급 최대 0.681/0.581).
- 상대 기준은 **여기서 오히려 틀립니다**. 중복이 많은 공간은 자기 분포가 올라가, 중복이 가장 많을 때 유난해 보이지 않게 됩니다.
- 내장 알고리즘은 이 판정에서도 관문에 걸립니다. 거의 같은 글 0.505–0.889와 어휘 함정 0.449–0.572의 **범위가 겹쳐** 분리되지 않습니다.

1-0. 수집과 정리

- 수집함은 소유자당 하나인 **평범한 공간**입니다(`spaces.is_inbox` , 부분 유니크 인덱스). 검색·임베딩·Dream·연결이 전부 그대로 동작하므로 하위 계층이 두 번째 종류의 저장소를 알 필요가 없습니다. 삭제는 거부합니다 — 지

우면 다음 수집이 조용히 새로 만들면서 이전 것을 잃습니다.

- `/capture` 는 공간 id를 받지 않습니다. 그게 요점입니다. 재시도 키 허용 목록에 포함되어 오프라인에서도 담기고 재시도가 사본을 만들지 않습니다.
- `MoveNote` 는 출발지와 목적지 **양쪽의 쓰기 권한**을 각각 확인합니다. 연결은 공간에 속하고 양 끝이 그 안에 있어야 하므로 옮기는 노트는 연결을 가져갈 수 없고, 삭제한 개수를 반환해 호출자가 **미리** 알릴 수 있게 합니다.
- `SuggestSpaces` 는 임베딩이 의미를 판별한다고 측정됐을 때만 `basis=meaning` 을 주장하고, 아니면 `basis=recent` 로 물러섭니다. 어휘 중복을 이해인 것처럼 내놓는 것보다 한계를 밝히는 편이 낫습니다.
- `LoadEmbeddings` 는 저장된 벡터만 읽습니다. 벡터는 공간 조회 시 만들어지므로, 비교 대상 노트의 공간을 조회하지 않으면 벡터가 없고 모든 코사인인 0이 됩니다 — 정렬처럼 보이지만 정렬이 아닙니다. 후보는 공간별로 나눠 부르지 않고 **한 번에** 모아 해석합니다. 호출이 나뉘면 각기 다른 저장 알고리즘이 선택되어 순위를 매기려는 공간들 사이에서 점수가 비교 불가능해집니다.

0-8. 의미 줌: 멀리서 볼 때 보이는 것

- 0.45 줌 아래에서는 노트 대신 **뭉친 것들**을 그립니다. 기존 줌 범위(0.15~2.2)에서 포스트잇 글자가 읽히지 않게 되는 지점입니다. 계층은 **두 단계**입니다 — 군집 위에 주제, 주제 위에 지식 섬을 만들 데이터가 없고, 없는 계층을 지어내면 근거 없는 구조를 사실처럼 보여 주게 됩니다.
- 노트 **25개 미만이면 전환하지 않습니다**. 요약은 정보 과부하를 푸는 기능이고, 과부하가 없는 캔버스에서는 같은 것의 더 나쁜 뷰입니다(6개짜리 공간이 `fitView` 때문에 스스로를 요약하던 실측 문제).
- **판정할 수 없으면 배치로 묶습니다**. 낮은 줌에서 군집은 노트를 *대체*하므로, 어휘로 묶인 군집을 그렇게 보여 주면 워크스페이스에 대해 자신 있게 틀린 그림을 줍니다. umm은 공간 도구이고 노트 위치는 사람이 결정한 데이터이므로, 배치 기준 군집은 지어낸 구조가 아니라 실재하는 구조를 보고합니다. `ThoughtCluster.Basis` 가 어느 쪽인지 싹고, 화면도 실선/점선으로 구분합니다 — 다른 주장이기 때문입니다.
- 배치 군집은 **단일 연결**입니다. 한 줄로 늘어놓은 생각은 쌍의 사슬이 아니라 하나의 흐름이고, 사람들은 실제로 그렇게 배치합니다. 거리 기준 120px은 절대값이고 그래도 됩니다 — 유사도와 달리 캔버스에는 사람이 작업하는 고정된 척도가 있습니다(기본 노트 260×180).
- 덩어리는 **멤버들이 차지한 땅을 그대로 덮습니다**. 줌아웃이 레이아웃을 재배치하면 umm에서 가장 중요한 사용자 데이터를 잃습니다. 어느 덩어리에도 속하지 않은 노트는 그대로 남습니다 — 숨기면 워크스페이스가 실제보다 정돈된 것처럼 보입니다.
- 라벨 크기는 화면 픽셀이 아니라 **캔버스 단위**입니다. 이 뷰는 0.45 아래에서만 나타나므로 쓰인 크기의 절반 이하로 그려집니다.

1-1-0. 판정 기준은 설정이고, 바닥은 설정이 아닙니다

- 유사도 문턱은 전부 `app_settings` 의 `intelligence` 영역에서 옵니다(`store.IntelligenceSettings`). 기본값은 측정해서 정한 상수 그대로라 바꾸지 않으면 동작이 같습니다.
- 기준은 표준편차 단위이므로 임베딩 백엔드를 바꿔도 같은 뜻을 유지합니다. 저장된 값이 범위를 벗어나면 **그 항목만** 기본값으로 되돌아갑니다 — 잘못된 설정 하나가 전체를 멈추지 않습니다.
- 저장 시에는 조용히 보정하지 않고 **거부**합니다. 표준편차 칸에 40을 넣은 관리자에게는 그게 표준편차가 아니라고 말해야지, 화면과 다르게 동작해서는 안 됩니다.
- `Semantic` 판정에서 `Discrimination > 0` 은 **설정**이 아닙니다. 관문의 두 하한은 낮출 수 있지만, 어휘가 뜻을 앞서는 백엔드는 어떤 값으로도 통과하지 못합니다. 관문 전체가 그 상태를 막으려고 존재합니다.

- 품질 리포트는 캐시되는데 판정은 기준에 의존하므로, 캐시된 리포트를 다시 재지 않고 다시 판정합니다 (`QualityReport.WithBars`). 측정값은 기준과 무관하므로 정확하고 무료이며, 설정 변경을 보지 못한 다른 인스턴스도 같은 숫자에서 같은 결론에 도달합니다.
- 설정 캐시는 30초입니다. 캔버스 로드와 검색마다 읽히므로 매번 DB를 왕복하면 설정 테이블이 핫 패스에 올라옵니다.

1-1-1a. 게이트웨이 자동 탐색

- 제대로 된 임베딩을 붙이는 데 남아 있던 마찰은 **모델 이름을 모른다는** 것이었습니다. `GET /admin/ai-gateway/discover` 가 알려진 주소에 OpenAI 호환 `/v1/models` 를 물어 응답한 게이트웨이와 모델 목록을 돌려줍니다.
- 조사 주소는 바이너리에 고정되어 있고 요청에서 읽지 않습니다. 읽는다면 이 화면이 서버가 닿을 수 있는 모든 곳을 조사하는 수단이 되고, umm 자신의 사이드카를 찾는 데는 그런 것이 필요 없습니다.
- 후보 응답은 신뢰하지 않습니다. 그 포트에 다른 것이 떠 있을 수 있으므로 본문 크기를 제한하고, HTML·오류·잘못된 JSON에서 모델이 나오지 않는지 테스트로 고정합니다.
- `likelyEmbedding` 은 이름 기반 힌트입니다. 게이트웨이는 서빙하는 모든 모델을 나열하고 무엇이 임베딩인지 응답에 없습니다. 채팅 모델 사이에서 후보를 위로 올릴 뿐이고, 판정은 `/admin/ai-gateway/test` 와 품질 측정이 합니다.
- 후보는 선언 순서대로 보고합니다. 가장 빨리 응답한 호스트가 아니라 umm이 문서에 적어 둔 사이드카가 먼저 와야 합니다.

1-1-2. 임베딩 품질을 운영자에게 노출

- 설정한 임베딩 모델이 실제로 쓰이고 있는지는 제품 밖에서 알 방법이 없었습니다. 게이트웨이가 죽거나 모델명에 오타가 나면 umm은 조용히 로컬 임베딩으로 폴백하고, 화면은 그대로 "연관 생각"과 "의미상 유사"를 말합니다.
- `GET /api/v1/admin/embedding-quality` 가 설정된 백엔드로 라벨 데이터를 직접 임베딩해 판별력·쌍별 정확도·주제 분리·최근접 동일 주제와 함께 세 가지 상태를 구분해 반환합니다: 미설정(`semantic=false`), 설정했으나 폴백 중(`fellBack=true`), 정상(`semantic=true`).
- provider는 `EmbeddingProvider` 가 아니라 설정에서 직접 해석합니다. 회로 차단기가 열려 있는 동안 로컬로 치환된 값을 보여 주면, 장애를 조사하러 온 운영자에게 정상처럼 보이기 때문입니다.
- 결과는 백엔드 identity 기준 10분 캐시입니다. 설정 화면을 열 때마다 22쌍(44문장)을 운영자의 추론 서버로 다시 보내지 않기 위해서입니다.
- 문장쌍만으로는 부족합니다. 쌍 데이터는 "이 둘이 같은 뜻인가"를 묻지만 연관 생각과 군집은 "이 공간에서 무엇이 무엇과 묶이는가"를 묻습니다. 라벨된 4개 주제 16문장으로 **최근접 이웃이 같은 주제인 비율**을 함께 재며, `semantic` 판정에 포함합니다. 실제로 쌍 지표 1위 모델이 이 항목과 중단 군집 테스트에서 떨어졌습니다.
- 지표는 **모델 선택의 순위표가 아닙니다**. 집계 점수가 높아도 특정 워크스페이스에서 더 나은 군집이 나온다는 보장은 없습니다. 모델마다 틀리는 문장이 다르고, 메모가 적을수록 한 번의 실수가 결과를 지배합니다. 후보 비교는 `docs/admin-guide.md` 에 있습니다.
- 임베딩 모델을 umm 이미지에 번들하지 않는 이유는 `CGO_ENABLED=0` 정적 단일 바이너리입니다. 대신 `compose.yaml` 의 `embeddings` 프로파일로 OpenAI 호환 추론 서버를 옆에 세웁니다. 폐쇄망에서도 모델을 한 번만 반입하면 됩니다.

1-1-3. 뜻과 출처를 나눠 기록하는 연결

- `note_edges.relation` 은 요청 본문에서 온 자유 텍스트였고 출처 칸이 없었습니다. 실측 결과 일반 사용자가 `relation='dreamed'` 로 **Dream**이 발견했다고 주장하는 엷지를 만들 수 있었고, 5000자 문자열도 저장돼 캔버스가 라벨로 렌더링했습니다.
- 모델링 오류는 하나였습니다. 한 칸이 뜻과 만든 주체를 함께 담고 있었습니다. `relation` 은 검사된 어휘 (`related·supports·contradicts·refines·expands·follows`)로 뜻만 담고, `origin` (`manual·agent·dream·development·import·auto`)이 주체를 담습니다.
- `origin` 은 요청 본문에서 읽지 않습니다. 저장소의 단일 쓰기 경로 `createEdge` 가 호출하는 코드에서 인자로 받으므로, 핸들러가 나중에 JSON을 그대로 바인딩해도 주체를 고를 수 없습니다. 웹은 `manual`, MCP는 `agent` 입니다.
- 어휘 밖의 값은 `related` 로 조용히 바꾸지 않고 400과 `allowedRelations` 로 거부합니다. 사용자가 설명하지 않은 연결을 기록하고 실수를 감추지 않기 위해서입니다.
- `confidence` 는 `origin='auto'` 일 때만 값을 갖도록 제약이 걸려 있습니다. 점수 없는 추측은 순위를 매길 수 없고, 사람이 그 선의 점수는 아무도 만들지 않은 숫자입니다.
- `UNIQUE(source,target)` 을 `UNIQUE(source,target,relation)` 으로 바꿔 한 쌍에 여러 뜻이 공존합니다. 기계 추천이 사람이 그 선을 덮지 못합니다. 역방향 탐색용 `(target_note_id, relation)` 인덱스를 추가했습니다 — 이전에는 전체 스캔이었습니다.
- 엷지를 읽는 모든 경로가 `origin` 과 `confidence` 를 실어야 합니다. v0.11.0에서 `Backlinks` 하나를 빠뜨렸고 아무것도 실패하지 않았습니다 — 필드가 빈 값으로 도착했을 뿐이라 화면에는 출처 없는 연결이 나왔습니다. OpenAPI는 처음부터 `Edge.origin` 을 필수로 요구하고 있었으므로 명세가 맞고 구현이 틀렸는데 확인하는 것이 없던 경우입니다. 지금은 테스트가 고정합니다.
- MCP의 `get_connections` 가 이 그래프를 에이전트에게 열어 줍니다. `get_related_notes` 가 임베딩으로 비슷해 보이는 것을 찾는 데 반해, 이쪽은 실제로 기록된 연결을 방향·뜻·출처와 함께 돌려주므로 에이전트가 자기가 쓴 부분을 식별할 수 있습니다.
- 마이그레이션 010의 데이터 이동 경로는 `scripts/graph-migration-drill.sh` 가 실제 데이터로 앞뒤로 굴러 검증합니다. `migrate-dry-run.sh` 는 빈 스키마에서 돌기 때문에 이 부분을 건드리지 않습니다.
- 백필의 한계는 남습니다. 릴리스 전에 위조된 `dreamed` 엷지는 진짜와 구별할 정보가 기록된 적이 없어 `origin='dream'` 으로 함께 옮겨집니다.

1-1-4. 자동 연결과 그 관문

- `SuggestLinks` 는 공간의 모든 짝을 채점해 그 공간 자신의 분포에서 두드러지는 것들을 `origin='auto'` 엷지로 기록합니다. 사라지는 목록이 아니라 실제 엷지인 이유는, 지금 답하지 않으면 없어지는 제안은 없느니만 못하기 때문입니다.
- 백엔드가 의미를 판별한다고 측정되지 않으면 실행을 거부합니다. `MeasureEmbeddingQuality(...).Semantic` 이 관문입니다. 기본 로컬 알고리즘은 뜻보다 어휘를 높게 보므로(쌍별 정확도 4.2%), 그 위에서 제안하면 단어가 겹치는 무관한 생각들을 이어 놓게 됩니다.
- `outcome` 은 `suggested / no-candidates / backend-not-semantic / too-few-notes` 네 가지입니다. 조용한 결과에도 종류가 있고, "찾지 못했다"와 "판단을 거부했다"는 다른 대응을 부릅니다. 거부 경로에서도 `edges` 는 항상 배열입니다.
- `confidence` 는 확률이 아니라 그 공간의 평균에서 몇 표준편차 위인지를 0.5~0.99로 옮긴 값입니다. 화면에도 "두드러진 정도"로 적습니다. 측정이 뒷받침하는 주장은 그것뿐입니다.

- 거절은 `link_dismissals` 에 남습니다. 자동 연결은 이미 연결된 짝을 건너뛰므로, 추천을 지우면 건너뛴 근거까지 사라져 다음 실행에서 그대로 다시 올라왔습니다(실측). 짝은 방향과 무관하게 정규화해 저장합니다. **사람이 그 연결을 지울 때는 거절을 기록하지 않습니다** — 나중에 다시 제안받고 싶을 수 있습니다.
- 한 번 실행에 최대 12개입니다. 제안이 쌓이면 사람은 전부 무시합니다.

1-2. 인덱스를 타는 하이브리드 검색

- 키워드 조건은 `단어마다 하나의 ILIKE` 를 AND로 묶은 형태로 생성됩니다. 각 조건이 `pg_trgm` GIN 인덱스를 타고 PostgreSQL이 비트맵을 교집합합니다.
- 인덱스는 `(title || ' ' || content)` 표현식 위에 만들어집니다. 코드의 표현식과 한 글자라도 달라지면 조용히 순차 스캔으로 되돌아가므로, 두 값이 일치하는지 검사하는 테스트를 함께 둡니다.
- 의미 점수 후보는 최근 문서로 상한을 둡니다. 큰 워크스페이스에서 한 번의 검색이 전체 스캔이 되지 않게 하기 위해서입니다.

2. 낙관적 동시성 제어 (Optimistic Concurrency Control)

- 각 생각 노드는 단조 증가하는 `version` 번호를 가집니다.
- 다중 사용자가 동시에 작업하더라도 버전 충돌을 감지하고 409 Problem Details에 최신 서버 메모를 포함합니다. 브라우저는 내 변경과 서버 변경을 나란히 보여 주고 선택·병합한 뒤 새 버전으로 저장합니다.
- 댓글 해결·재개·삭제 transaction은 대상 댓글을 update lock으로, 활성 생각·공간과 비소유자의 현재 membership 권한을 shared lock으로 고정된 다음 mutation과 `space_events` ·webhook outbox를 함께 commit 합니다. 멤버 제거와 `edit/manage → view` 강등은 이 transaction 뒤에서 직렬화되므로 point-in-time 권한 확인으로 협업 상태를 뒤늦게 바꾸지 못합니다.
- 오프라인 변경은 PWA local queue에 먹통 키와 함께 보관됩니다. 재연결 시 안전한 순서로 replay하고 같은 메모의 연속 PUT은 마지막 상태로 합칩니다. 모든 browser storage 읽기·쓰기·삭제는 공용 예외 경계 안에서 수행하며, quota 또는 권한 오류가 나면 mutation을 저장했다고 보고하지 않습니다. Canvas는 메모별 마지막 서버 응답 또는 성공적으로 기록된 queue 상태를 별도로 유지하고, 비내구성 autosave가 실패했으며 그 뒤 새 편집도 없다면 그 상태로 즉시 복원합니다. 요청 중 시작한 더 최신 편집은 객체 세대로 구분해 먼저 되돌리지 않고 자신의 직렬화된 저장 결과로 확정합니다. 인증된 queue owner는 메모리에도 유지해 저장소 접근이 차단되어도 앱 초기화와 상태 조회를 계속하고, 읽을 수 없거나 손상된 기존 queue는 빈 배열로 덮어쓰지 않습니다. 인증 복구 가능성이 있는 일반 401/403은 queue를 멈추지만, 공간에서 제거되었거나 생각이 삭제되어 댓글 해결·삭제를 더는 적용할 수 없는 경우 서버가 `comment-mutation-forbidden` 403을 반환하므로 그 항목만 사유를 알리고 제거한 뒤 이후의 독립 변경을 계속 처리합니다. 서비스 워커의 / 앱 셀 캐시는 성공한 `text/html` 탐색 응답만 갱신하므로 manifest·asset JSON·SVG를 주소창에서 직접 열어도 오프라인 셀이 비-HTML 응답으로 바뀌지 않습니다. 8자 content hash가 붙은 `assets/` bundle만 1년 immutable이고, manifest·service worker·아이콘·asset manifest 같은 고정 URL은 `no-cache` 로 매 배포 재검증합니다.

3. 분산 큐 & DB 트랜잭션 (SKIP LOCKED)

- 외부 메시지 브로커(RabbitMQ, Redis 등) 없이 PostgreSQL의 `FOR UPDATE SKIP LOCKED` 를 활용하여 Dream 백그라운드 작업을 분산 워커 간 중복 없이 안전하게 임대(Lease) 처리합니다.
- 생성된 Dream은 `dream_notes.note_id IS NULL` 인 검토 후보로 먼저 저장됩니다. Today·검토함·상세와 source query는 Dream 소유권뿐 아니라 현재 공간 owner/member 권한을 함께 확인하므로 공유가 회수되면 파생 본문·공간명·원본도 응답에서 빠집니다. AI Assist와 자동 Dream 생성은 선택·선별된 원본 note 행, 원본 공간

행과 membership을 잠그며 재생성·발전은 Dream 행과 Dream 공간까지 같은 lease에 포함합니다. 공간 ID를 정렬된 순서로 잠근 채 외부 Gateway 호출 종료까지 유지하므로, 공유 회수가 먼저 확정되면 쿼터 예약을 호출 전에 취소하고 AI lease가 먼저 시작되면 회수가 호출 종료까지 기다립니다. 긴 lease는 request pool을 빌리지 않는 인스턴스당 2-slot PostgreSQL 용량으로 제한되어 세 번째 호출은 연결을 점유하지 않고 대기합니다. 자동 생성에서 품질을 통과한 Dream·source·알림도 이 transaction에서 함께 확정되어 point-in-time 조회 뒤 권한 변경으로 본문이나 부분 후보가 남지 않습니다. 채택 요청은 같은 방식으로 현재 공간·편집 membership을 잠근 뒤 메모와

`dreamed` 연결선을 한 번만 생성합니다. 채택 후 발전 결과도 같은 권한·Dream 행 잠금 아래 새 메모와 `expanded` 연결선을 원자적으로 만들며, 동일 본문 재시도는 기존 결과를 반환합니다.

- Dream 노출·숨김·선호 피드백은 목록 응답 시점이 아니라 브라우저 `IntersectionObserver` 에서 카드가 50% 이상 보인 시점에 기록됩니다. 서버는 Dream 행과 현재 공간 membership을 같은 transaction에서 잠근 뒤 상태와 개인화 점수를 함께 확정하므로 권한 회수 뒤 보관한 ID로 피드백을 만들 수 없습니다. 알림의 `resourceType/resourceId` 는 Dream 카드 또는 공유 공간 딥링크로 해석됩니다.
- 공간·메모의 `ai_excluded` 정책은 Scheduler 자격 계산과 AI Assist·Dream Gateway 직전의 최종 source lease에서 모두 다시 적용되어, 설정 변경 이후의 신규 AI 처리에서 제외됩니다.

4. Daily review와 하이브리드 탐색

- `/today` 는 14일 이상 검토하지 않은 생각, 사용자가 지정한 다시 보기, 연결선이 없는 생각, 대기 Dream, 최근 댓글을 현재 권한과 삭제 상태 범위 안에서 집계합니다. 알림 목록과 unread count도 note 알림의 현재 생각 행과 Dream 알림의 현재 Dream 행을 join해 실제 공간 접근권한을 같은 기준으로 적용하며, legacy 알림의 비어 있는 `resource_space_id` 를 신뢰 경계로 사용하지 않습니다. `note_reviews` 가 공유 메모의 검토·고정 상태를 사용자별로 분리하고, `review_digest` 는 협업 활동 query만 제어해 기본 검토 신호를 제거하지 않습니다.
- 검색은 완전 로컬 192차원 feature-hash cosine 점수와 제목·본문·공간 키워드 점수를 결합합니다. 응답용 본문 일부가 잘려도 전체 본문의 키워드 일치 플래그를 점수에 보존하며, 공간·종류·수정 시각 필터와 opaque cursor를 지원합니다.
- 백링크는 실제 `note_edges` 의 incoming/outgoing 방향을 반환하며 연관 생각은 의미 유사성을 보완합니다.

5. 자동화·관측성·공급망

- 도메인 변경, PostgreSQL SSE log, 활성 구독별 `webhook_deliveries` outbox는 같은 트랜잭션에서 커밋됩니다. 세 워커가 가용 dispatch slot을 확보한 뒤 `FOR UPDATE SKIP LOCKED` 로 대기 또는 lease가 만료된 처리 항목을 선점하므로 프로세스 재시작과 순간 부하 뒤에도 전달을 이어갑니다. 각 워커는 `claimed_at` 세대가 정확히 일치하는 delivery와 subscription·owner·space·membership을 잠그고 실제 HTTP 응답, terminal delivery 전환, payload 삭제, subscription counter 갱신까지 같은 transaction에서 확정합니다. 권한 회수·사용자 비활성화·구독 중지가 먼저 끝나면 외부 전송을 시작하지 않고, 전달 lease가 먼저 시작되면 정책 변경이 그 종료까지 기다리므로 point-in-time 확인 뒤 캡처한 payload가 새 정책을 넘어 나가지 않습니다. HMAC 서명, SSRF 재검증, 제한 재시도와 자동 중지를 적용하며, at-least-once 전달 시도의 중복은 delivery UUID로 수신 측이 제거합니다. terminal metadata는 30일 보존합니다. 외부 오류는 잘못된 UTF-8을 제거하고 byte 상한 안의 완전한 rune 경계에서 잘라 PostgreSQL 상태 전환과 실패 횟수 갱신을 안정적으로 끝냅니다.
- HTTP 계층은 low-cardinality route pattern으로 Prometheus count-latency를 기록합니다. `/api/v1/metrics` 는 관리자 브라우저 세션 또는 명시적인 `metrics:read` API 키만 허용하고, 일반 세션의 wildcard와 관리자 소유 일반 키는 운영 지표 권한으로 해석하지 않습니다. 표준 OTLP 환경변수가 있을 때만 OpenTelemetry exporter를 초기화합니다.

- 태그 파이프라인은 offline image archive와 SPDX SBOM을 만들고 checksum 및 GitHub artifact attestation을 발급합니다.

6. 남용 방지와 수평 확장의 경계

- 로그인 실패 횟수와 AI 하루 사용량은 **PostgreSQL**에서 셉니다. 비밀번호 확인부터 실패 기록 또는 session commit까지 주소·계정별 advisory transaction lock 안에서 처리하고 모든 DB 작업은 같은 연결을 사용하므로, 인스턴스가 여러 대이거나 pool 상한이 1이어도 병렬 추측이 설정한 실패 상한을 넘어가지 않습니다.
- 일반 API 요청 한도는 인스턴스별 인메모리 토큰 버킷입니다. 요청마다 데이터베이스를 왕복하면 막으려는 부하보다 비용이 커지기 때문이며, 그 결과 실패 상한은 $\text{설정값} \times \text{인스턴스 수}$ 가 됩니다. 전역으로 지켜져야 하는 한도는 앞의 두 가지이고, 그것은 데이터베이스에서 셉니다.
- 보안 섹션의 whole-object PUT은 설정별 PostgreSQL advisory transaction lock을 얻은 뒤 최신 행을 읽고, 구버전 payload가 생략한 다섯 남용 방지 필드만 병합합니다. OIDC `client_secret` 과 AI Gateway `api_key` 의 마스킹 저장도 같은 방식으로 비밀 필드를 생략해 최신 암호문을 transaction 안에서 병합하며, master-key 회전은 같은 두 lock을 정해진 순서로 잡고 재암호화합니다. 동시 관리자 저장과 롤링 업그레이드·키 회전이 겹쳐도 먼저 확인한 한도나 새 암호문을 stale snapshot으로 지우지 않으며 명시한 0은 omission과 구분합니다.
- 계정별 로그인 잠금 임계값은 주소별의 3배입니다. 아이디를 아는 사람이 남의 계정을 임의로 잠글 수 있는 상태를 피하기 위한 의도적인 비대칭입니다.
- 만료된 세션·OAuth state·재시도 기록·로그인 실패 기록은 15분마다 정리됩니다. 모든 인스턴스가 동시에 실행해도 안전합니다.
- `scripts/multi-instance-smoke.sh`가 이 세 가지(교차 인스턴스 이벤트 전달, 교차 인스턴스 맥등 재시도, 공유 로그인 잠금)를 CI에서 실제로 검증합니다.